

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

Dr.kesavan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I __S.JAYA SHREE(192524146)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.JAYA SHREE

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Query Processing Steps & Query Optimizer Decision-Making

Query processing in a DBMS follows these stages:

1. **Parsing & Translation** – The SQL query is checked for syntax/semantic correctness and translated into an internal representation (relational algebra expression / parse tree).
2. **Query Optimization** – The optimizer generates multiple equivalent execution plans and estimates their cost (I/O, CPU, memory) using statistics (table size, indexes, selectivity, histograms).
3. **Code Generation / Plan Selection** – The plan with the lowest estimated cost is chosen and converted into an executable form (query execution plan).
4. **Execution** – The execution engine runs the plan and returns results.

How the optimizer decides the best plan:

- It enumerates alternative access paths (full scan vs. index scan) and join orders/algorithms (nested loop, sort-merge, hash join).
- It uses a **cost model** based on statistics stored in the system catalog (number of tuples, page count, index availability).
- It applies **transformation rules** (e.g., pushing selection/projection down before joins) to reduce intermediate result sizes.
- Finally, it picks the plan with minimum estimated total cost (I/O + CPU cost).

2. Heuristic vs. Cost-Based Optimization

Aspect	Heuristic Optimization	Cost-Based Optimization
Basis	Fixed rules/transformations (e.g., do selections early)	Actual cost estimates using statistics
Speed	Fast, low overhead	Slower, more computation-intensive
Accuracy	May not always produce the optimal plan	Produces near-optimal plan based on data distribution
Data dependency	Independent of actual data statistics	Heavily dependent on up-to-date statistics

When to use each:

- **Heuristic optimization** is preferred when statistics are unavailable/unreliable, or for simple queries where rule-based transformation (like predicate pushdown) is clearly beneficial regardless of data size.
- **Cost-based optimization** is preferred for complex queries with multiple joins, where different access paths/join orders can produce vastly different performance, and where the system maintains reliable statistics (common in modern RDBMS like Oracle, PostgreSQL).

In practice, most modern optimizers use heuristics first to reduce the search space, then apply cost-based estimation among the remaining candidate plans.

3. Database Connectivity: JDBC vs. ODBC

Implementation approach in a real project:

1. Load the appropriate driver.
2. Establish a connection using a connection string (URL, credentials).
3. Create a statement/prepared statement object.
4. Execute queries and process the ResultSet.
5. Close connections properly (often using connection pooling in production, e.g., HikariCP).

JDBC vs ODBC:

Aspect	JDBC	ODBC
Platform	Java-specific, platform-independent	Language-independent, mostly Windows-based (C/C++ origin)
Interface	Pure Java API	C-based API, requires driver manager
Performance	Slightly slower due to abstraction layer	Generally faster due to native calls
Usage	Used in Java/web applications	Used in legacy/desktop applications, works with many languages
Driver types	Type 1–4 drivers (Type 4 = pure Java, most efficient)	ODBC drivers per DBMS vendor

In a real project, JDBC is preferred for Java-based enterprise applications (e.g., Spring Boot apps connecting to MySQL/PostgreSQL), while ODBC is used when connecting from non-Java tools like Excel, or legacy C/C++ applications.

4. ACID Properties with Real-World Examples

- **Atomicity** – A transaction executes fully or not at all. *Example:* Bank fund transfer — debit from Account A and credit to Account B must both succeed or both fail; otherwise money is created or lost.
- **Consistency** – The database moves from one valid state to another, respecting all constraints. *Example:* An airline seat-booking system must never allow total booked seats to exceed available seats.
- **Isolation** – Concurrent transactions do not interfere with each other's intermediate states. *Example:* Two customers booking the last movie ticket simultaneously — isolation ensures only one succeeds, without seeing the other's uncommitted change.
- **Durability** – Once committed, changes persist even after a system crash. *Example:* Once an ATM withdrawal is confirmed and printed as a receipt, the deduction must survive a subsequent power failure.

5. Concurrency Control Problems & 2PL

- **Dirty Read** – A transaction reads data written by another uncommitted transaction; if that transaction rolls back, the read value becomes invalid.
- **Lost Update** – Two transactions read the same data and update it concurrently; one update overwrites the other, losing changes.

- **Unrepeatable Read** – A transaction reads the same row twice within itself and gets different values because another transaction modified and committed changes in between.

How Two-Phase Locking (2PL) solves these: 2PL requires every transaction to acquire all necessary locks before releasing any (Growing Phase), and once it starts releasing locks, it cannot acquire new ones (Shrinking Phase).

- Prevents **dirty reads** by requiring shared/exclusive locks before read/write, blocking access to uncommitted data (especially in Strict 2PL).
- Prevents **lost updates** by ensuring exclusive locks are held until commit, so concurrent writers must wait.
- Prevents **unrepeatable reads** by holding read locks until the transaction completes, blocking others from modifying the data mid-transaction.

2PL guarantees **conflict serializability**, ensuring a schedule's outcome is equivalent to some serial execution.

6. Deadlock in DBMS: Detection & Prevention

Deadlock occurs when two or more transactions wait indefinitely for locks held by each other, forming a circular wait.

Deadlock Detection Algorithm:

- The system builds a **Wait-For Graph (WFG)**, where an edge $T_i \rightarrow T_j$ means T_i is waiting for a lock held by T_j .
- Periodically, the system checks for a **cycle** in this graph.
- If a cycle is found, a deadlock exists, and the system selects a **victim transaction** to abort (often based on minimal cost — least work done, fewest locks held) and rolls it back to break the cycle.

Prevention Strategies:

1. **Wait-Die Scheme** – An older transaction may wait for a younger one; a younger transaction requesting a lock held by an older one is aborted (dies) and restarted.
2. **Wound-Wait Scheme** – An older transaction "wounds" (forces rollback of) a younger transaction holding the needed resource; a younger transaction simply waits for an older one.
3. Other strategies: **lock ordering** (always acquire locks in a predefined global order) and **timeout-based** approaches (abort a transaction if it waits beyond a threshold).

7. Immediate Update vs. Deferred Update Recovery

Aspect	Immediate Update	Deferred Update
When DB is modified	Changes written to disk before commit	Changes applied to disk only after commit
Undo needed?	Yes, since uncommitted changes may be on disk	No undo needed, only redo
Redo needed?	Yes	Yes

Log requirement	Must log before-image (old value) and after-image	Must log only after-image (new value)
Recovery complexity	More complex (needs UNDO + REDO)	Simpler (only REDO)

When each is preferred:

- **Immediate Update** is preferred in systems needing high concurrency/performance where writes should reach disk early (typical in most commercial RDBMS with buffer management), accepting more complex recovery logic.
- **Deferred Update** is preferred in simpler or smaller systems where minimizing recovery complexity matters more than write-ahead performance, since it avoids the need to undo any operation (nothing is written until commit).

8. Shadow Paging

Shadow Paging avoids traditional logging by maintaining two page tables: the **current page table** (reflecting in-progress changes) and the **shadow page table** (reflecting the last consistent, committed state).

- Modifications are made to new copies of pages; the current page table is updated to point to these new pages.
- The shadow table remains untouched until commit.
- On commit, the shadow table is replaced by the current table (an atomic switch).
- On failure/crash, the system simply reverts to the shadow table, discarding uncommitted changes.

Advantages:

- No need for UNDO/REDO logging — recovery is fast (just discard current table).
- No overhead of writing log records for every update.

Limitations compared to log-based recovery:

- **Data fragmentation** — old pages become garbage and require garbage collection.
- **Poor support for concurrent transactions** — since it works on the granularity of the whole database page table, it's hard to support fine-grained concurrent commits.
- **Commit overhead** — updating the page table pointer can still be costly for large databases.
- Difficult to support **fine-grained rollback** of a single transaction among many concurrent ones, unlike log-based methods.

9. Serializability & Conflict Serializability Testing

A **serializable schedule** is a schedule of concurrent transactions whose final effect is equivalent to some serial (one-at-a-time) execution of those same transactions, ensuring correctness despite concurrency.

Conflict Serializability is tested using a **precedence graph (serialization graph)**:

1. Create a node for each transaction in the schedule.
2. For every pair of conflicting operations (both access the same data item, at least one is a write) where operation of T_i precedes operation of T_j , draw a directed edge $T_i \rightarrow T_j$.

3. **If the resulting graph has no cycles**, the schedule is conflict serializable, and a valid serial order can be obtained via topological sort of the graph.
4. **If a cycle exists**, the schedule is NOT conflict serializable.

Example: If T1 writes X before T2 reads X, and T2 writes Y before T1 reads Y, edges $T1 \rightarrow T2$ and $T2 \rightarrow T1$ form a cycle — hence not serializable.

10. 2PL Variations: Basic, Strict, and Rigorous

- **Basic 2PL** – Transactions acquire locks in a growing phase and release them in a shrinking phase, but locks can be released **before commit**, once no longer needed. This guarantees conflict serializability but **not** recoverability (can lead to cascading rollbacks if a transaction aborts after releasing a lock that another transaction already used).
- **Strict 2PL** – All **exclusive (write) locks** are held until the transaction commits or aborts; other locks may be released earlier. This prevents dirty reads/cascading rollbacks, ensuring recoverability, and is widely used in practice.
- **Rigorous 2PL** – **All locks (both shared and exclusive)** are held until commit/abort. This is stricter than Strict 2PL, simplifies concurrency control implementation (transactions can be serialized in the order they commit), but reduces concurrency slightly further.

Most commonly used: Strict 2PL is the most widely used in commercial DBMSs (e.g., Oracle, SQL Server) because it strikes a balance — it prevents cascading aborts and ensures recoverability while still allowing shared locks to be released early for better concurrency, unlike Rigorous 2PL which is more conservative than necessary for most use cases.